

Lab Exercise 2: Building a Mini Online Shopping Database in MySQL

Instructor: Sanzhar Kusdavletov

1 Lab Objectives

By the end of this lab, you should be able to:

- create a relational database for a small online shopping system
- choose suitable MySQL data types
- use primary keys and foreign keys
- apply SQL constraints such as NOT NULL, UNIQUE, DEFAULT, and CHECK
- insert and query data across related tables
- understand referential integrity in a practical scenario

2 Scenario

You are asked to design a simple database for a small online shop. The system should store:

- customer information
- product information
- orders placed by customers
- items included in each order

3 Software Requirements

- MySQL Server
- MySQL Workbench

4 Task 1: Create the Database

Create a database for this lab.

```
CREATE DATABASE online_shop_lab;  
USE online_shop_lab;
```

5 Task 2: Create the Customers Table

Create a table called `Customers`.

Requirements:

- `customer_id` must be the primary key
- `customer_id` must auto increment
- `customer_name` cannot be `NULL`
- email must be unique
- country should have a default value of 'Kazakhstan'

Suggested solution:

```
CREATE TABLE Customers (  
    customer_id INT AUTO_INCREMENT,  
    customer_name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    country VARCHAR(50) DEFAULT 'Kazakhstan',  
    PRIMARY KEY (customer_id)  
);
```

6 Task 3: Create the Products Table

Create a table called `Products`.

Requirements:

- `product_id` must be the primary key
- product name cannot be `NULL`
- price must be greater than 0
- stock cannot be negative

Suggested solution:

```
CREATE TABLE Products (  
    product_id INT AUTO_INCREMENT,  
    product_name VARCHAR(120) NOT NULL,  
    price DECIMAL(10,2) NOT NULL,  
    stock INT DEFAULT 0,  
    PRIMARY KEY (product_id),  
    CHECK (price > 0),  
    CHECK (stock >= 0)  
);
```

7 Task 4: Create the Orders Table

Create a table called `Orders`.

Requirements:

- `order_id` must be the primary key
- `customer_id` must reference the `Customers` table
- order date cannot be `NULL`
- status should have a default value

Suggested solution:

```
CREATE TABLE Orders (  
    order_id INT AUTO_INCREMENT,  
    customer_id INT NOT NULL,  
    order_date DATE NOT NULL,  
    status VARCHAR(30) DEFAULT 'Pending',  
    PRIMARY KEY (order_id),  
    FOREIGN KEY (customer_id)  
        REFERENCES Customers(customer_id)  
);
```

8 Task 5: Create the OrderItems Table

Create a table called `OrderItems`.

Requirements:

- each row should connect an order with a product
- quantity must be greater than 0
- use foreign keys
- use a composite primary key

Suggested solution:

```
CREATE TABLE OrderItems (  
    order_id INT,  
    product_id INT,  
    quantity INT NOT NULL,  
    PRIMARY KEY (order_id, product_id),  
    FOREIGN KEY (order_id)  
        REFERENCES Orders(order_id),  
    FOREIGN KEY (product_id)  
        REFERENCES Products(product_id),  
    CHECK (quantity > 0)  
);
```

9 Task 6: Insert Sample Data

Insert a few customers.

```
INSERT INTO Customers (customer_name, email, country)
VALUES
('Aigerim', 'aigerim@mail.com', 'Kazakhstan'),
('Daniel', 'daniel@mail.com', 'UK'),
('Ali', 'ali@mail.com', 'Kazakhstan');
```

Insert a few products.

```
INSERT INTO Products (product_name, price, stock)
VALUES
('Laptop', 350000.00, 10),
('Mouse', 5000.00, 50),
('Keyboard', 12000.00, 25),
('Monitor', 80000.00, 8);
```

Insert orders.

```
INSERT INTO Orders (customer_id, order_date, status)
VALUES
(1, '2026-03-10', 'Pending'),
(2, '2026-03-10', 'Shipped');
```

Insert order items.

```
INSERT INTO OrderItems (order_id, product_id, quantity)
VALUES
(1, 1, 1),
(1, 2, 2),
(2, 3, 1),
(2, 4, 1);
```

10 Task 7: Test the Constraints

Try the following statements and observe what happens.

Test 1: NOT NULL

```
INSERT INTO Customers (email)
VALUES ('new@mail.com');
```

Test 2: UNIQUE

```
INSERT INTO Customers (customer_name, email)
VALUES ('Test User', 'ali@mail.com');
```

Test 3: CHECK

```
INSERT INTO Products (product_name, price, stock)
VALUES ('Invalid Product', -100.00, 5);
```

Test 4: FOREIGN KEY

```
INSERT INTO Orders (customer_id, order_date)
VALUES (100, '2026-03-10');
```

Test 5: Composite Primary Key

```
INSERT INTO OrderItems (order_id, product_id, quantity)
VALUES (1, 1, 3);
```

Write a short explanation for each error message.

11 Task 8: Retrieve Data

Show all customers

```
SELECT * FROM Customers;
```

Show all products

```
SELECT * FROM Products;
```

Show all orders

```
SELECT * FROM Orders;
```

Show all order items

```
SELECT * FROM OrderItems;
```

12 Task 9: Join the Tables

Use JOIN to display useful business information.

Query 1: Show customer orders

```
SELECT c.customer_name, o.order_id, o.order_date, o.status
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id;
```

Query 2: Show products in each order

```
SELECT oi.order_id, p.product_name, oi.quantity
FROM OrderItems oi
JOIN Products p ON oi.product_id = p.product_id;
```

Query 3: Show full order details

```
SELECT c.customer_name, o.order_id, p.product_name, oi.quantity,
       p.price
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
JOIN OrderItems oi ON o.order_id = oi.order_id
JOIN Products p ON oi.product_id = p.product_id;
```

13 Task 10: Challenge Exercise

Extend the database by adding one of the following:

- a) a `category` column in the `Products` table
- b) a `phone_number` column in the `Customers` table
- c) a `payment_method` column in the `Orders` table

Then insert data into the new column.

14 Reflection Questions

Answer the following questions:

1. Why is `customer_id` used as a primary key instead of customer name?
2. Why do we use a separate `OrderItems` table instead of storing products directly in the `Orders` table?
3. What is the purpose of foreign keys?
4. What type of problems can happen if constraints are not used?
5. Why is a composite primary key useful in `OrderItems`?

15 Expected Learning Outcome

After completing this lab, you should be able to design and implement a simple relational database for a realistic business case and test whether your constraints correctly protect the data.